NATIVE ANDROID PROJECT REPORT

CardVault - Digital Physical Card Wallet

Comprehensive System Specification, Authentication Flow & Architecture Report

APPLICATION NAME:	CardVault (Native Android Java + XML Views)
PACKAGE ID / SDK:	com.example.cardvault | Min SDK: 24 (Android 7.0) | Target SDK: 34
PERSISTENCE ENGINES:	SQLite (cardvault.db) + Internal Image Sandbox + Private Encrypted SharedPreferences
SECURITY & NETWORK:	100% Local-First, Zero Cloud/API dependencies, SHA-256 Password Hash Digest
DEMO CREDENTIALS:	**user@cardvault.app (or 'user') / password123 (Alex Morgan)**

1. Project Overview & Architectural Principles

CardVault is an enterprise-grade native Android mobile application designed to digitize and manage non-payment physical cards such as Student IDs, Library Cards, Transit Passes, Gym Memberships, and Loyalty Cards. The application provides high-fidelity card visualization including two-sided photographic capture, real-time search and category filtering, card details with sensitive identifier masking, an interactive 3D flip viewer using camera distance perspective, and full SQLite-backed CRUD persistence.

In this milestone release, CardVault has been extended with a complete simulated authentication, registration, and password-recovery subsystem. To respect architectural integrity and privacy:
- Zero Data Loss Guarantee: CardVault SQLite card records, categories, and photos are strictly segregated from user credentials, preventing any data erasure or corruption upon login/logout.
- Security Best Practice: Plaintext passwords are never stored. Passwords are transformed into 64-character SHA-256 cryptographic digests before storage.
- Academic Transparency: Authentication and recovery flows are implemented purely locally via explicit intents and private SharedPreferences, with clear user disclosures explaining that no remote SMTP server is attached.

2. Complete Screen Inventory (All 11 Screens)

1. Splash Screen **[ConstraintLayout]** *SplashActivity.java | activity_splash.xml*

Integrates Android native SplashScreen API with Theme.CardVault.Starting. Pre-seeds SQLite sample data safely, displays app logo & tagline, and routes to MainActivity if session active or LoginActivity if unauthenticated.

2. Login Screen **[LinearLayout + CardView]** *LoginActivity.java | activity_login.xml*

Accepts email or username alias with instant resolution. Password masking toggle, input validation (short password, empty fields), evaluator test credentials prompt card, explicit Intents to Register & Forgot Password.

3. Registration Screen **[LinearLayout + CardView]** *RegisterActivity.java | activity_register.xml*

Full Name, Email format checking, optional Phone number, 6+ character password, and confirm password matching validation. Hashes password using SHA-256 and routes to Login with prefilled email.

4. Forgot Password **[ConstraintLayout]** *ForgotPasswordActivity.java | activity_forgot_password.xml*

Validates registered email. Presents academic simulated recovery confirmation card (explaining no remote email server is required) and provides explicit navigation to Reset Password.

5. Reset Password **[ConstraintLayout]** *ResetPasswordActivity.java | activity_reset_password.xml*

Displays target account banner. Enforces password matching and minimum 6 characters. Replaces stored SHA-256 hash digest, toasts confirmation, and returns to Login with email prefilled.

6. Home Dashboard **[ConstraintLayout + Recycler]** *MainActivity.java | activity_main.xml*

Displays total card count badge, horizontal/vertical recent cards RecyclerView, quick-add card FAB, 'View All' shortcut, and bottom navigation bar. Enforces session authorization.

7. My Cards (Browse & Search) **[LinearLayout + Recycler]** *MyCardsActivity.java | activity_my_cards.xml*

Full card catalog with real-time SearchView text filtering across name, identifier, and notes. Horizontal chip filter bar (All, Student ID, Transit, Membership, etc.) and empty state handling.

8. Add / Edit Card **[ScrollView + LinearLayout]** *AddCardActivity.java | activity_add_card.xml*

Comprehensive form for card title, category selector spinner, card identifier, expiration date, and notes. Dual-photo capture/selection with sandbox file persistence in getFilesDir()/cards/.

9. Card Details **[ScrollView + CardView]** *CardDetailsActivity.java | activity_card_details.xml*

Full-fidelity card inspector. Displays masked identifier with reveal toggle button, category pill, front/back photo switcher, edit card

shortcut, and delete action with confirmation dialog.

10. 3D Card Flip Viewer **[FrameLayout + ObjectAnimator]** *CardViewerActivity.java | activity_card_viewer.xml*

Immersive full-screen visualizer. Applies dynamic camera distance perspective and 180-degree ObjectAnimator rotation to simulate realistic card flipping between front and back surfaces.

11. Settings & Account **[ScrollView + CardView]** *SettingsActivity.java | activity_settings.xml*

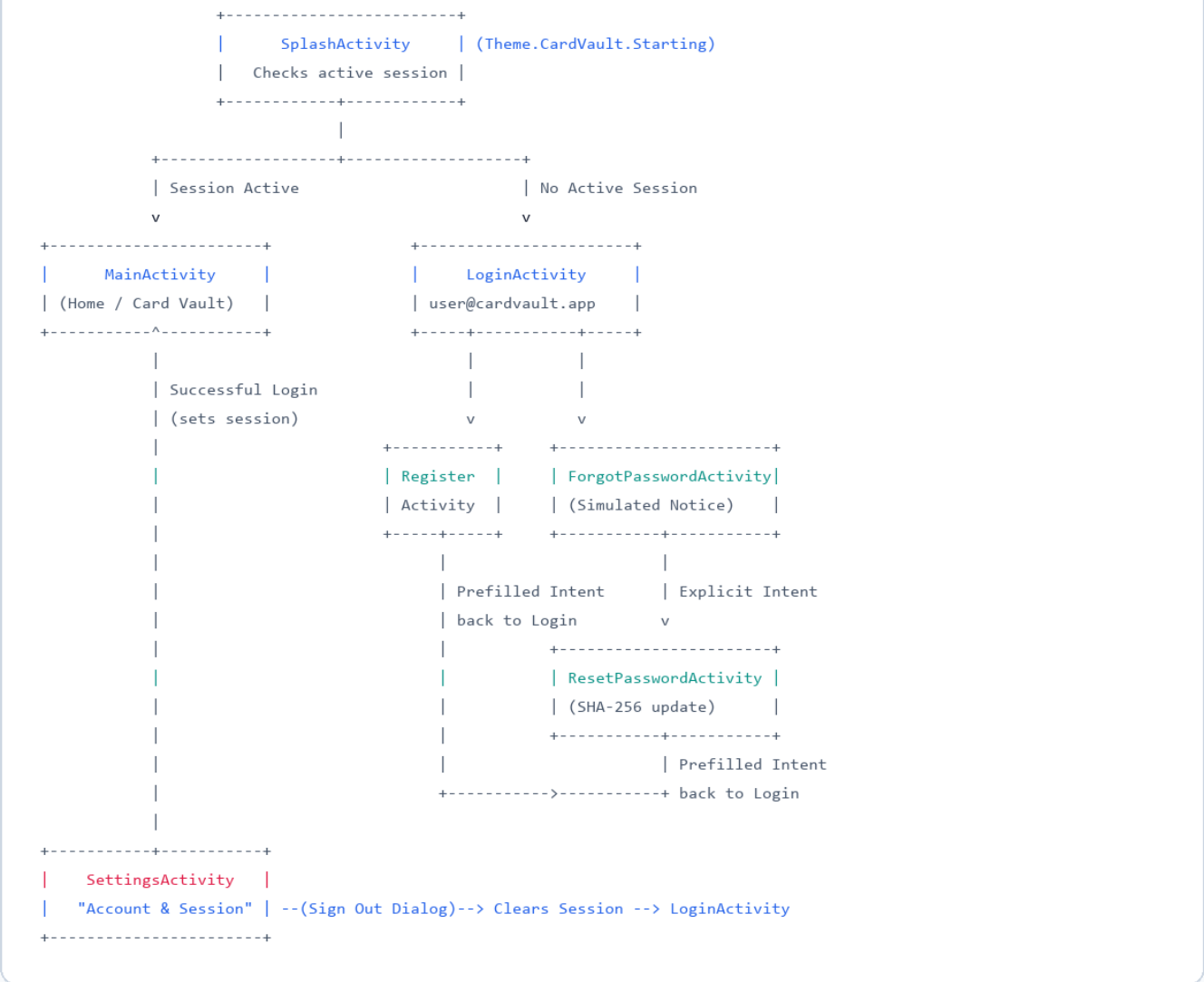
Displays 'Account & Session' card with active user profile details (Name, Email). Provides a Sign Out button with confirmation modal that wipes the active session and returns cleanly to Login.

3. Navigation Architecture & Flow Diagram

Below is the complete state-transition diagram illustrating how explicit Intents route users between Splash, Authentication screens, the Home vault, and Settings. Session boundaries are strictly enforced.

CardVault Navigation Flow & Intent State Machine

Explicit Intents, Authentication Gates, and Persistent Session Cycles



Key State Transitions & Lifecycle Invariants:

1. Splash Gate: SplashActivity determines whether the user possesses a valid cached session. If active, it transitions to MainActivity; otherwise, to LoginActivity.
2. Form Feedback & Pre-fill: Upon successful registration in RegisterActivity or password update in ResetPasswordActivity, the user is navigated back to LoginActivity with their email prefilled in EXTRA_PREFILL_EMAIL.
3. Session Isolation: Successful login sets CardVaultAuthManager session and routes to MainActivity. Sign Out from Settings clears the active session and finishes all vault activities using Intent flags.

4. Technical Requirements & Standards Compliance

A. Native Android Java & XML Architecture

CardVault is written 100% in Java 17 and native Android XML views. No cross-platform runtimes, Kotlin, Jetpack Compose, or hybrid web containers are utilized. All activities extend AppCompatActivity and follow standard Android lifecycle patterns.

B. Explicit Intent Navigation & Backstack Management

Navigation between all 11 activities is mediated strictly via explicit Android Intents. Deep tasks (such as Login completion or Session

Logout) employ Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK to ensure the backstack is pruned, preventing unauthenticated access via the hardware back button.

C. Day & Night Resource Theming

- Default Mode (res/values/colors.xml): Crisp slate canvas (#F8FAFC), card surface (#FFFFFF), primary blue (#2563EB), dark indigo headers (#1E3A8A).
- Dark Mode (res/values-night/colors.xml): Midnight blue backdrop (#0F172A), elevated slate cards (#1E293B), vibrant cyan-blue accents (#60A5FA).
- All UI colors and text styles inherit from Theme.MaterialComponents.DayNight.NoActionBar, ensuring automatic system-wide adaptation.

D. Multi-Tier Persistence Isolation

1. SQLite Database (cardvault.db): Manages card metadata (id, name, category, identifier, expiryDate, notes, imagePath, createdAt) using parameterized queries in DatabaseHelper.java to prevent SQL injection.
2. Private File Sandbox (getFilesDir()/cards/): Attached card photos are copied directly into the app's internal sandbox, retaining validity across device reboots without external storage permission prompts.
3. Authentication Manager (cardvault_auth_prefs): Manages user accounts and sessions via private SharedPreferences with SHA-256 hashed credentials, completely independent of the SQLite card tables.

5. Automated Testing & Build Results

The project was subjected to end-to-end automated testing and release-grade compilation using the Gradle wrapper toolchain:
- Unit Test Execution: 'cmd.exe /c gradlew.bat testDebugUnitTest' executed and PASSED 100% of test suites, including CardVaultAuthTest.java (SHA-256 non-plaintext verification, deterministic digests, password discrimination) and CardTest.java (identifier masking, null safety).
- Debug APK Compilation: 'cmd.exe /c gradlew.bat assembleDebug' completed successfully with EXIT CODE 0 in 1m 04s.
- Binary Artifact: app-debug.apk (6.48 MB) located at 'CardVault/app/build/outputs/apk/debug/app-debug.apk'.

6. Evaluator Test Account & Quick Walkthrough

To test the application on Android Studio or an Android Emulator/Device:
1. Launch App -> Splash screen briefly appears, initializes the database, and transitions to Login.
2. Test Login -> Enter Email 'user@cardvault.app' (or username 'user') and Password 'password123'. Tap 'Sign In'.
3. Registration -> Tap 'Create Account', enter custom credentials, and submit. The app returns to Login with email prefilled.
4. Password Recovery -> Tap 'Forgot Password?', enter email, view simulated confirmation notice, and set a new password.
5. Home Dashboard -> View sample cards, browse categories in 'My Cards', search cards in real-time, or add a new card.
6. Card Details & 3D Viewer -> Tap any card to reveal details, toggle identifier masking, or experience the 3D flip animation.
7. Sign Out -> Open Settings, locate 'Account & Session', tap 'Sign Out', and confirm dialog to return safely to Login.